TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie

Ecole Supérieure Privée Technologies & Ingénierie

Type d'épreuve	: ■ Devoir	☐ Examen	
Enseignant	: **Ameni Mejri, Faten Ayachi, Jihene Bokri, Aymen Ben Saada**		
Matière	: **Algorithmique et structures de données I**		
Année Universitaire	: 2020 -2021	**Semestre**	: 1
Classe	: **TIC-1**		
Documents	: ☐ Autorisés	■ Non autorisés	
Date	: 13/01/2021	**Durée :**	1h30
Nombre de pages	: **2**		

Exercice 1 (4 pts)

Le code creux consiste à coder un tableau sans les zéros qu'il comporte et qui constituent une information inutile pour certains calculs. Ce code comporte un tableau tel que chaque élément est formé par une coordonnée non nulle et par sa position dans le tableau initial.

Par exemple le tableau [1, 2, 0, 5, 8, 0, 0, 0, 7] donne en code creux [(1,1) (2,2), (5,4), (8,5), (7,9)]

Nous proposons la structure de donnée tableau d'enregistrements pour stocker le code creux du tableau. Soient les deux types suivants :

```
type element = enregistrement
                indice : entier
                valeur : réel
           fin
type tabCodeCreux = tableau[1..50] de element
type tabReels = tableau[1..100] de réels
```

- Écrire une procédure permettant de coder un tableau de réels en code creux.

EXERCICE 1 : (6 pts)

On considère la suite définie par

U0 = 2

Un = 3 * Un-1 - 1

1) Ecrire sous forme itérative une fonction qui étant donné un entier m, calcule l'indice du premier terme de la suite supérieur ou égal à m.

2) Ecrire sous forme récursive cette même fonction.

Exemple : si m=30 la fonction retournera 3 car
U0=2
U1= 3*2-1=5
U2=3*5-1=14
U3=3*14-1=41 > m

Exercice 2 (10 pts = 1 + 2.5 + 2,5 + 4)

On se propose de gérer les avions d'une compagnie aérienne. Chaque avion est caractérisé par un identifiant (entier), un modèle (chaine de caractères), le nombre de places disponibles (entier) et la consommation du kérosène par kilomètre (réel).

1. Faites la déclaration de la structure de données **Avion**.
2. Soit les deux procédures supposées définies :

   `procédure saisieAvion (resultat AV : Avion)` qui permet de saisir les informations d'un avion donné.

   `procédure afficheAvion (donnee AV : Avion)` qui permet d'afficher les informations d'un avion donné.

   On souhaite créer un tableau de type **`T_Avion`** qui stocke les descriptions des avions disponibles pour une compagnie aérienne. Faites la déclaration de **`T_Avion`**.

   Ecrire la procédure **`saisie_T_Avion`** qui permet la saisie des informations sur N avions dans un tableau.

3. Ecrire une fonction qui retourne le modèle de l'avion ayant le plus grand nombre de places disponibles.
4. Ecrire une procédure qui permet de :
   a. Saisir N avions.
   b. Afficher les informations sur les N avions.
   c. Afficher le modèle de l'avion qui a le plus grand nombre de places.
   d. Afficher la consommation en kérosène d'un avion donné connaissant son identifiant ainsi que le nombre de kilomètres à parcourir entrés par l'utilisateur.

- Un **prix** de type réel

1. Faites les déclarations nécessaires en algorithmique afin de créer un tableau **Bib** qui peut contenir jusqu'au 1000 livres. **(1pt)**
2. Ecrire une procédure **remplirBib** qui permet de remplir le tableau des livres en donnant la taille effective en paramètre avec le tableau. **(1 pt)**
3. Ecrire une procédure **afficheBib** qui permet d'afficher toutes les informations des livres disponibles. **(1pt)**
4. Ecrire une fonction **existeLivre** qui permet de vérifier si le titre d'un livre passé en paramètre existe dans la bibliothèque. Elle retourne -1 si le titre n'existe pas, sinon elle retourne l'indice de la première occurrence trouvée. **(1 pt)**
5. Ecrire une fonction **nbLivresChersRv** qui calcule le nombre de livres de la bibliothèque dont le prix dépasse **100D**. **(2pt)**
6. Ecrire un **algorithme principal** (main) qui permet de **(2 pts)**
   1. Demander à l'utilisateur le nombre des livres de la bibliothèque.
   2. Remplir le tableau des livres.
   3. Afficher tous les livres pour vérifier le remplissage du tableau.
   4. Afficher le nombre de livres dont le prix dépasse 100D.
   5. Demander à l'utilisateur le titre du livre à chercher. Afficher son prix s'il existe, sinon afficher un message indiquant qu'il n'existe pas.

Exercice 3 :

Soit les définitions suivantes :

TYPE tableauNonTrie = **tableau** [1 ... N] d'entier.
TYPE tableauTrie = **tableau** [1 ... N] d'entier.

Le tri de comptage détermine, pour chaque élément x (entier) d'un tableau non trié A (de type tableauNonTrie), le nombre d'éléments inferieurs à x. Cette information sera utilisée pour placer x dans sa position dans le tableau de sortie B (de type tableauTrie). Par exemple, si 17 éléments sont inférieurs à x, alors x sera placé à la position 18 dans le tableau de sortie B.

1. Ecrire une procédure ***countingSort(A : tableauNonTrie, var B : tableauTrie)*** qui permet d'implémenter le tri de comptage en prenant en considérations les répétitions possibles pour un entier du tableau A. Exemple pour A=<6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2>, le tableau de sortie B sera B=<0, 0, 1, 1, 2, 3, 3, 4, 6, 6>. **(5 pts)**

2. Illustrer les états du tableau de sortie B pour le tableau non trié A suivant : **(1pt)**

A	6	0	2	4